MATH DIRECTED READING PROGRAM / MEETING 2

From the Chain Rule to Training a Network

Self-contained guided reading and problem sheet | Revision 3.0

Aim. Separate three tasks: evaluate a model, differentiate its loss, and update its parameters. Recognize familiar multivariable calculus in the terms *forward pass*, *backpropagation*, and *gradient descent*.

Background and pacing. Matrix multiplication, partial derivatives, and the chain rule. Local reviews repeat the needed network notation. Work through 2.1–2.3 first; 2.4 can be a second discussion. Extensions 2.E1–2.E2 are optional. One module need not fit into one meeting.

Using the self-checks. After each numbered problem, try its short self-checks with the worked solution covered. Definitions may stay open. Give a reason, not only a yes/no answer; these checks guide discussion rather than add a timed test.

2.1 What does it mean to update parameters?

REVIEW: A LOSS IS A FUNCTION OF ADJUSTABLE NUMBERS

A **model** f_θ is a function with adjustable parameters θ. In a **forward pass**, we hold θ fixed and compute a prediction $\widehat{y} = f_\theta(x)$ for an input x. A target y supplies a desired output. A **loss** $\ell(\widehat{y}, y)$ measures disagreement; for a scalar prediction, one choice is $\frac{1}{2}(\widehat{y} - y)^2$.

When training, the examples are fixed and the parameters vary. We can list all parameter entries as $\theta \in \mathbb{R}^P$. For a differentiable training loss $L(\theta)$, its **gradient** is the column vector $\nabla_\theta L = (\partial L/\partial \theta_j)_{j=1}^P$.

For a fixed direction v, the chain rule gives

$$\frac{d}{dt} L(\theta + tv)\Big|_{t=0} = \nabla L(\theta)^\top v.$$

A **gradient-descent step** is $\theta^+ = \theta - \eta \nabla L(\theta)$, where the chosen number $\eta > 0$ is the **learning rate**. An **optimizer** is a rule for updating parameters; gradient descent is one such rule. The learning rate is a **hyperparameter**: a setting of the procedure, not a weight learned by this update.

Worked example. For $L(\theta) = \frac{1}{2}(\theta - 3)^2$, at $\theta = 1$ the derivative is -2. With $\eta = \frac{1}{2}$, the update gives $\theta^+ = 2$. The loss changes from 2 to $\frac{1}{2}$.

Guided questions

(a) Substitute $v = -\nabla L(\theta)$ in the directional derivative. If the gradient is nonzero, why does a sufficiently small positive step decrease a differentiable loss?

(b) For the worked example, show $\theta^+ - 3 = (1 - \eta)(\theta - 3)$. Starting away from 3, determine which $\eta > 0$ strictly decrease the loss. What happens at $\eta = 2$ and $\eta = 3$?

(c) Does a zero gradient, by itself, prove that a general loss is minimized? Compare the functions t^2 and $-t^2$ at $t = 0$.

Translate back into machine-learning language. Explain the difference between computing a derivative and taking a training step. Neither a large step nor a zero gradient automatically certifies successful training.

SELF-CHECK 2.1

1. For $L(\theta) = \frac{1}{2}(\theta - 2)^2$ at $\theta = 0$, compare one step with learning rate $\eta = 1/2$ and one with $\eta = 3$. Which decreases the loss, despite both using the negative gradient?
2. Classify these actions: evaluate $f_\theta(x)$, compute $\nabla_\theta L$, replace θ by $\theta - \eta \nabla_\theta L$. Which changes the parameter values?

2.2 Backpropagation in a scalar network

REVIEW: THE CHAIN RULE, ORGANIZED BY COMPUTATIONS

A **computational graph** records elementary calculations and which intermediate values each calculation uses. A forward pass evaluates them in dependency order. For a scalar loss ℓ, write $\bar{u} = \partial \ell / \partial u$ for the derivative with respect to an intermediate scalar u.

A **backward pass** starts at the loss and applies the chain rule in reverse order, reusing saved forward values. This procedure is **backpropagation**. More generally, **automatic differentiation** computes derivatives by combining derivative rules for the operations in a program; backpropagation is its reverse-mode application to a scalar loss. No parameter update is part of this differentiation step.

Use one input x and target y, with trainable scalars w, b, v, c:

$$a = wx + b, \qquad h = \mathrm{ReLU}(a), \qquad \widehat{y} = vh + c, \qquad \ell = \tfrac{1}{2}(\widehat{y} - y)^2.$$

Here a is a **pre-activation**, and h is a **hidden activation**. The **ReLU activation function** is $\mathrm{ReLU}(a) = \max(0, a)$, with derivative 1 for $a > 0$ and 0 for $a < 0$. It is not differentiable at zero; all required calculations avoid that point.

Worked start. Take $x = 2$, $y = 1$, $(w, b, v, c) = (1, -1, 3, 0)$. The forward values are $a = 1$, $h = 1$, $\widehat{y} = 3$, $\ell = 2$. Start the backward pass with $\bar{\widehat{y}} = \widehat{y} - y = 2$.

Guided questions

(a) From $\widehat{y} = vh + c$, compute $\partial \widehat{y}/\partial v$, $\partial \widehat{y}/\partial c$, and $\partial \widehat{y}/\partial h$. Use the chain rule to find $\bar{v}, \bar{c}, \bar{h}$.

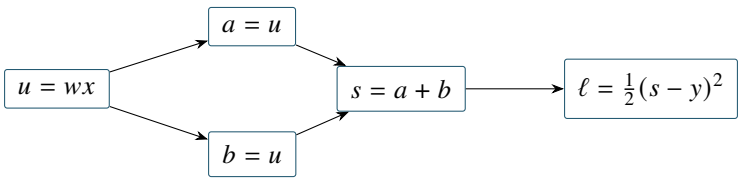
(b) Propagate through $h = \mathrm{ReLU}(a)$, then $a = wx + b$, to obtain $\bar{a}, \bar{w}, \bar{b}$. Give all four parameter derivatives numerically for the worked start.

(c) You can also compute $\bar{x}$. Why does differentiating with respect to x not make the fixed training input a trainable parameter?

(d) Keep the parameters and target fixed but change x to 0. Repeat the forward pass. Which derivatives become zero because this hidden unit has negative pre-activation? Is the derivative with respect to c also zero?

REVIEW: TWO USES OF ONE COMPUTED VALUE

Derivative contributions from different uses add. The two identity nodes below are copies of the same computed value u; they are not independently trainable variables. Arrows show which earlier value a calculation uses.



(e) For this separate graph, take $w = x = 1$ and $y = 0$. Compute its forward values. Annotate the two backward paths into u and add their contributions to $\bar{u}$. Find $\bar{w}$ and check it by differentiating $\frac{1}{2}(2wx - y)^2$ directly. What derivative would be lost if only one path were counted?

Translate back into machine-learning language. Say what is moving backward: derivatives of the loss, not predictions, inputs, or updated parameter values.

SELF-CHECK 2.2

1. In the shared-use graph of part (e), why are the two contributions to $\bar{u}$ added, not averaged or overwritten? What would counting only one path do to $\bar{w}$?
2. In the original ReLU network, a particular input makes the hidden pre-activation negative. Does that make every parameter derivative zero, including the output bias? Explain by following the computational paths.

2.3 The same calculation with matrices

REVIEW: VECTOR AND MATRIX DERIVATIVES

For a scalar loss ℓ and a column vector $u \in \mathbb{R}^r$, $\bar{u} = \nabla_u \ell$ is a column vector. For a matrix $A \in \mathbb{R}^{r \times s}$, $\bar{A} = \nabla_A \ell$ has the *same shape* as A, with entry $\bar{A}_{ij} = \partial \ell / \partial A_{ij}$.

For $u = g(x)$, the **Jacobian** is the matrix J_g with entries $(J_g)_{ij} = \partial u_i / \partial x_j$. The chain rule reads $\bar{x} = J_g(x)^\top \bar{u}$.

For equal-length vectors, $p \odot q$ denotes coordinatewise multiplication. The **outer product** $pq^\top$ is the matrix with entries $p_i q_j$.

An affine layer. Let $u = Ax + b$, where $x \in \mathbb{R}^s$, $u, b \in \mathbb{R}^r$, and $A \in \mathbb{R}^{r \times s}$. In coordinates,

$$u_i = \sum_{j=1}^{s} A_{ij} x_j + b_i.$$

Suppose later operations have already supplied $\bar{u}$. The layer's backward computation must produce $\bar{x}$, $\bar{A}$, $\bar{b}$.

One worked entry. Varying A_{12} changes u_1 at rate x_2 and leaves all other u_i unchanged. Hence

$$\frac{\partial \ell}{\partial A_{12}} = \sum_i \frac{\partial \ell}{\partial u_i} \frac{\partial u_i}{\partial A_{12}} = \bar{u}_1 x_2.$$

Guided questions: the local backward rule

(a) Repeat the worked calculation for A_{ij} and for b_i. Assemble your entries into $\bar{A} = \bar{u}x^\top$ and $\bar{b} = \bar{u}$, checking both shapes.

(b) Compute $\partial \ell / \partial x_j$ by summing over i. Identify the matrix expression $\bar{x} = A^\top \bar{u}$. Why is the transpose necessary with column gradients?

(c) For $h = \mathrm{ReLU}(a)$ applied coordinatewise, set $r_i = 1$ when $a_i > 0$ and $r_i = 0$ when $a_i < 0$. Show that $\bar{a} = \bar{h} \odot r$. Here all a_i are assumed nonzero.

A SMALL NUMERICAL CHECK

Let $A = \begin{pmatrix} 1 & 2 \\ -1 & 0 \end{pmatrix}$, $x = (3, 4)^\top$, $b = 0$, and suppose $\bar{u} = (2, -1)^\top$. Your rules give $\bar{A} = \begin{pmatrix} 6 & 8 \\ -3 & -4 \end{pmatrix}$ and $\bar{x} = (3, 4)^\top$. These derivatives use the supplied $\bar{u}$.

Translate back into machine-learning language. Explain "the gradient with respect to a weight matrix" as an array of familiar partial derivatives, not as an additional kind of network layer.

2.3 / COMPOSE THE LOCAL BACKWARD RULES

Use the one-hidden-layer network

$$a = W_1 x + b_1, \quad h = \mathrm{ReLU}(a), \quad \widehat{y} = W_2 h + b_2, \quad \ell = \tfrac{1}{2}\|\widehat{y} - y\|_2^2,$$

where $\|v\|_2^2 = \sum_i v_i^2$ and

$$x \in \mathbb{R}^d, \quad W_1 \in \mathbb{R}^{H \times d}, \quad b_1, a, h \in \mathbb{R}^H, \quad W_2 \in \mathbb{R}^{m \times H}, \quad b_2, \widehat{y}, y \in \mathbb{R}^m.$$

The **hidden width** H counts the coordinates of h. The weight matrices and bias vectors are parameters; $a, h, \widehat{y}$ are computed values. Assume each pre-activation a_i is nonzero.

Guided questions: an entire backward pass

(d) Start at the loss: show $\overline{\widehat{y}} = \widehat{y} - y$. Apply the affine-layer rule to the output stage to obtain $\bar{W}_2, \bar{b}_2, \bar{h}$.

(e) Continue through ReLU, then through the first affine layer, obtaining $\bar{a}, \bar{W}_1, \bar{b}_1$. Check each gradient against the corresponding parameter's shape. Keep the same parameter values throughout this backward pass.

(f) Now take $d = H = 2$, $m = 1$, and

$$x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \; y = 1, \quad W_1 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \; b_1 = 0, \quad W_2 = \begin{pmatrix} 2 & -1 \end{pmatrix}, \; b_2 = 0.$$

Compute the forward values and all four parameter gradients. Which hidden unit contributes no gradient to its incoming weights at this input?

(g) Make a single gradient-descent update with $\eta = 0.05$ using the gradients from part (f). Update *all* parameter arrays from their old values, and only then recompute the prediction and loss. Did this particular step improve this particular example?

DO NOT UPDATE HALF-WAY THROUGH BACKPROPAGATION

The chain rule computes derivatives of the loss at one fixed parameter point. For example, using an already-updated W_2 to calculate $\bar{h}$ would mix parameter points. Store the gradients first, then apply the optimizer's update rule.

Translate back into machine-learning language. Describe the three passes through this task: calculate predictions; propagate derivatives; update parameters. Which is called a forward pass, which is backpropagation, and which is the optimizer step?

SELF-CHECK 2.3

1. For $u = Ax + b$, take $x = (2, -1)^\top$, $\bar{u} = (3, 4)^\top$, and $A \in \mathbb{R}^{2 \times 2}$. Compute $\bar{A}$ and $\bar{b}$ without re-deriving the full network. Why is $\bar{A}$ the same shape as A?
2. You have updated W_2 before calculating the derivative of the old loss with respect to W_1. Can the resulting arrays still be called the gradient at the old parameter point? Locate the mixed parameter values.

2.4 From one example to a mini-batch

SETUP: AVERAGE LOSS AND AVERAGE GRADIENT

For N training examples (x_s, y_s), let $\ell_s(\theta) = \ell(f_\theta(x_s), y_s)$ and

$$L(\theta) = \frac{1}{N}\sum_{s=1}^{N} \ell_s(\theta).$$

A **full-batch** step uses the gradient of this entire average. A **mini-batch** uses a smaller collection of examples to estimate it. In **stochastic gradient descent (SGD)**, an update uses a randomly selected example or mini-batch instead of the full dataset.

For this problem only, select indices $I_1, \ldots, I_B$ independently and uniformly from $\{1, \ldots, N\}$, allowing repeats, and set

$$g_B(\theta) = \frac{1}{B}\sum_{b=1}^{B} \nabla \ell_{I_b}(\theta), \qquad \theta^+ = \theta - \eta g_B(\theta).$$

The integer B is the **batch size**. An estimator is **unbiased** if its expected value equals the quantity being estimated. At a fixed θ, uniform selection means $\mathbb{E}[\nabla \ell_I(\theta)] = N^{-1}\sum_s \nabla \ell_s(\theta)$. Other sampling schemes require their own justification.

Worked average. If two example gradients at the same parameter point are 2 and -4, the full gradient is -1. A batch of size one returns either 2 or -4, not necessarily -1.

Guided questions

(a) Differentiate the finite sum defining L. Show $\mathbb{E}[g_B(\theta)] = \nabla L(\theta)$ under the stated sampling rule. No assumption that the training data themselves are random is needed here.

(b) With scalar parameter θ, take $\ell_1(\theta) = \frac{1}{2}(\theta - 1)^2$ and $\ell_2(\theta) = \frac{1}{2}(\theta + 1)^2$. Show $L(\theta) = \frac{1}{2}(\theta^2 + 1)$, minimized at $\theta = 0$.

(c) At $\theta = 0$, choose just example 1 and take a step with $0 < \eta < 1$. Check that its own loss decreases, but the full loss increases. Does this contradict unbiasedness of the sampled gradient?

(d) Explain why averaging the parameter gradients from several independent forward/backward calculations at the *same* parameter point gives the gradient of their average loss.

Translate back into machine-learning language. Translate "SGD gives a noisy gradient estimate" into a statement about randomly selecting terms of a finite sum. An unbiased derivative estimate does not mean every update decreases the full loss.

SELF-CHECK 2.4

1. Two example gradients at a fixed parameter point are 2 and -4. What is the average-loss gradient? If both dataset entries are duplicated and the loss is still averaged, does it change?
2. An SGD step improves the sampled example but worsens the dataset average. Does this alone contradict unbiasedness? State what quantity the expectation is taken over.

Optional extensions: checks and optimizer vocabulary

These are not prerequisites for Meeting 3. Complete the core backward pass before adding more update rules.

2.E1 / GRADIENT CHECKING IS DIFFERENT FROM BACKPROPAGATION

For the jth coordinate vector e_j and a small $\varepsilon > 0$, the central finite difference

$$\frac{L(\theta + \varepsilon e_j) - L(\theta - \varepsilon e_j)}{2\varepsilon}$$

approximates $\partial L/\partial \theta_j$ at a smooth point. Comparing it with a computed derivative is called **gradient checking**. Backpropagation instead applies the chain rule to the operations themselves. Extremely small ε can cause subtraction errors in finite-precision arithmetic.

Try it. Check the derivative with respect to w in 2.2 using $\varepsilon = 0.01$. Verify that neither perturbed pre-activation crosses zero. Why would a ReLU breakpoint complicate this comparison?

SELF-CHECK 2.E1

1. A finite-difference check crosses a ReLU breakpoint. Is disagreement with a chosen backward derivative evidence that the chain rule was implemented incorrectly? Explain the differentiability issue.

2.E2 / MOMENTUM AND ADAM: RECOGNIZE THE UPDATE RULES

An optimizer may store running information from earlier gradients. One common **momentum** convention is

$$m_t = \beta m_{t-1} + (1-\beta) g_t, \qquad \theta_{t+1} = \theta_t - \eta m_t, \quad m_0 = 0,$$

where $0 \le \beta < 1$ is a chosen decay factor. Other momentum conventions rescale m_t and the learning rate; they need not use the factor $(1-\beta)$.

Adam stores both a first-moment estimate m_t and a coordinatewise second-moment estimate v_t. In this standard form,

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, & v_t &= \beta_2 v_{t-1} + (1-\beta_2)(g_t \odot g_t), \\ \widehat{m}_t &= m_t/(1-\beta_1^t), & \widehat{v}_t &= v_t/(1-\beta_2^t), \\ \theta_{t+1} &= \theta_t - \eta\, \widehat{m}_t/(\sqrt{\widehat{v}_t} + \varepsilon). \end{aligned}$$

Here $m_0 = v_0 = 0$, $0 \le \beta_1, \beta_2 < 1$, and $\varepsilon > 0$. Products, roots, and divisions in the last two expressions are coordinatewise. A first moment concerns an average of values; a second moment concerns an average of squared values, not a centered variance.

Try it. If $g_t = g$ is constant, show by induction that $m_t = (1-\beta_1^t)g$ and $v_t = (1-\beta_2^t)(g \odot g)$. What do the correction factors remove in this special case? They do not turn every changing gradient sequence into an exactly unbiased population-moment estimate.

Translate back into machine-learning language. Momentum and Adam change the parameter-update rule. They do not replace the chain rule used to calculate the current gradient. The formulas alone are not convergence guarantees.

SELF-CHECK 2.E2

1. For a constant nonzero scalar gradient g, what do $\widehat{m}_t$ and $\widehat{v}_t$ equal? Is the second one a variance?
2. Does switching from SGD to Adam require a different derivative of the current network loss, or a different use of that derivative?

Discussion checkpoint and selected hints

READY TO MOVE ON WHEN...

Complete one forward pass, one backward pass, and one simultaneous parameter update on a small network, with every gradient at the same parameter point. Use 2.2–2.3 to explain the computation and 2.4 to distinguish an example gradient from an average-loss gradient. Drawing all computational paths matters more than memorizing optimizer formulas.
You may consult definitions and notation. Explain the choices without following a worked solution line by line. If a self-check exposes a gap, return to that problem's setup; optional extensions do not block progress.

CHECK YOUR TRANSLATIONS

A forward pass computes values at fixed parameters. Backpropagation computes loss derivatives at those parameters. An optimizer uses those derivatives to choose new parameters.
"Mini-batch" refers to a collection of examples, not a subset of a layer's coordinates. Unbiasedness is an expectation statement under a specified sampling rule, not a guarantee for one update.

Suggested discussion. Present the scalar calculation in 2.2, then use the affine-layer rule to explain one matrix gradient in 2.3. Point out the exact place where differentiation ends and optimization begins.

2.1(b). Compare $\frac{1}{2}(1-\eta)^2(\theta-3)^2$ with the old loss.

2.2(a)–(b). For $\widehat{y} = vh + c$, the three local derivatives are h, 1, and v. Multiply each by the arriving derivative $\overline{\widehat{y}}$.

2.3(d)–(e). Starting with $e = \widehat{y} - y$, use $\bar{W}_2 = eh^\top$, then $\bar{h} = W_2^\top e$. Continue through ReLU and the first affine stage.

2.3(f). The forward pre-activations are $(1, -2)^\top$, so only the first hidden coordinate passes a nonzero derivative through ReLU.

2.4(c). The sampled derivative is -1, so the new parameter is η. Substitute it into L and ℓ_1 separately.

Optional reading: reconnect with standard sources

[1] Prince, *Understanding Deep Learning* (2023). Author's book site. For 2.1 and 2.4: **§§6.1–6.2**, gradient descent and stochastic gradient descent. For 2.2–2.3: **§§7.2–7.4**, computing derivatives, toy example, and backpropagation algorithm. For the optional optimizers: **§§6.3–6.4**, momentum and Adam.

[2] Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning* (2020). Book site. **§§5.2–5.4**: gradients, Jacobians, matrix derivatives; **§5.6**: backpropagation and automatic differentiation; **§7.1**: gradient descent. Treat these as calculus review, not a second required textbook.

[3] Zhang, Lipton, Li, and Smola, *Dive into Deep Learning*. §5.3, Forward Propagation, Backward Propagation, and Computational Graphs; §12.5, Minibatch Stochastic Gradient Descent.

[4] Kingma and Ba, *Adam: A Method for Stochastic Optimization* (2015). Algorithm 1 and §3. Optional only: match the symbols in 2.E2 to the algorithm rather than reading the convergence discussion.

Readings supply context, not missing definitions. Exercises and numerical examples are original to this DRP. Prince locators refer to the 2023 first-edition organization; D2L locators refer to its 1.0.3 online edition. Use section titles when editions differ.